WEEK 3 PROJECT

CLASSIFICATION

Training Two Machine Learning Classifiers
and Comparing Their Performance

Project	Breast Cancer Diagnosis Classification (educational ML project)
Models	Logistic Regression and Random Forest
Tools	Python, Pandas, Matplotlib, Seaborn, Scikit-learn
Dataset	Scikit-learn Breast Cancer Wisconsin Diagnostic dataset
Deliverable	Cleaned workflow, two trained classifiers, evaluation metrics, comparison and conclusion

Prepared as a student-friendly project report with explanations, workflow, Python code, output interpretation and viva questions.

1. Project Overview

Task requirement: Train two classification models and compare them using proper evaluation metrics. The project follows the Week 3 compulsory task shown in the provided instructions.

What is classification? Classification is a supervised machine learning technique used to predict a category or class. For example, an email can be classified as spam or not spam. In this project, a model predicts whether a breast tumour record is labelled **malignant** or **benign**.

Project objective: Build two classifiers using the same dataset and the same train-test split, evaluate them fairly, and decide which model performs better on the selected test data.

Learning outcomes

• Understand a labelled dataset, features and target variable.

• Load and inspect data using Pandas and Scikit-learn.

• Separate training data from testing data to avoid evaluating on examples used for learning.

• Train Logistic Regression and Random Forest classifiers.

• Evaluate models with accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC.

• Compare the results and explain the strengths and limitations of each model.

Important: This is an educational demonstration using a public dataset. It is not a medical diagnostic tool and must never be used to make real healthcare decisions.

2. Dataset Description

The project uses the Breast Cancer Wisconsin (Diagnostic) dataset available directly through Scikit-learn. It contains measurements computed from digitised images of fine needle aspirates of breast masses.

Dataset detail	Description
Total records	569 patient sample records
Input features	30 numeric measurement features
Target classes	Malignant and Benign
Class counts	212 malignant; 357 benign
Missing values	The built-in Scikit-learn version has no missing values in its feature matrix
Source	Scikit-learn: load_breast_cancer()

Features are the input columns used by the model, such as mean radius, mean texture, mean perimeter and mean area. The **target** is the answer column the model learns to predict.

In Scikit-learn's original target coding, 0 represents malignant and 1 represents benign. The code uses the dataset's target names when displaying class labels, so the meaning is clear.

3. Tools and Technologies

Tool / library	Purpose
Python	Main programming language
Pandas	Data inspection and tabular operations

Tool / library	Purpose
NumPy	Numerical operations
Matplotlib	Charts and visualisations
Seaborn	Optional statistical visualisation library
Scikit-learn	Dataset loading, splitting, model training and evaluation
Jupyter Notebook / Google Colab	Environment for running the code

Installation (if needed):

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

4. Project Workflow

The complete process is shown below. Each step should be completed in order.

Step	Activity	Purpose
1	Import libraries	Prepare required Python tools
2	Load dataset	Get the labelled classification data
3	Explore data	Check rows, columns, classes and missing values
4	Split data	Create training and testing sets
5	Build models	Create Logistic Regression and Random Forest
6	Train models	Fit both models using training data only
7	Predict	Generate predictions for unseen test data
8	Evaluate	Calculate classification metrics and confusion matrices
9	Compare	Identify the stronger model for this test setup
10	Conclude	Summarise findings and possible improvements

5. Understanding the Two Classifiers

5.1 Logistic Regression

Logistic Regression is a supervised classification algorithm. Despite the word 'regression' in its name, it is commonly used for classification. It estimates the probability that an input belongs to a class and uses a threshold (usually 0.5) to assign a label.

Simple example: If the model estimates a 0.90 probability for the benign class, it predicts benign.

Advantages: Simple, fast, interpretable and often a strong baseline. **Limitations:** It may not capture complicated non-linear relationships unless suitable transformations or features are added.

5.2 Random Forest

Random Forest is an ensemble learning algorithm. It builds many decision trees using different samples and feature selections. For classification, the trees vote and the class with the most votes becomes the final prediction.

Simple example: If 150 out of 200 trees vote benign and 50 vote malignant, the forest predicts benign.

Advantages: Can learn non-linear patterns, handles feature interactions and usually requires little scaling. **Limitations:** Can be larger and less interpretable than a single tree, and may need tuning.

Why use two different models?

The two models learn in different ways. Logistic Regression provides a simple linear baseline, while Random Forest can learn more complex decision boundaries. Comparing them shows whether a more flexible model improves performance on this dataset.

6. Data Preparation

The dataset is loaded from Scikit-learn. The feature matrix is named **X** and the target labels are named **y**.

• **Train-test split:** 80% of records are used for training and 20% for testing.

• **Stratification:** stratify=y keeps approximately the same class proportion in both sets.

• **Random state:** random_state=42 makes the split reproducible.

• **Scaling:** StandardScaler is applied to Logistic Regression because feature scales differ. It is placed inside a Pipeline so the scaler is fitted only on training data. Random Forest does not require standard scaling for this example.

The test set is kept separate until evaluation. This gives a fairer estimate of how the models may perform on unseen records.

7. Evaluation Metrics

Metric	Meaning	How to interpret
Accuracy	Correct predictions divided by all predictions	Overall percentage correct; can mislead with imbalanced class
Precision	True positives divided by predicted positives	Of the records predicted positive, how many were correct?
Recall	True positives divided by actual positives	Of all actual positive records, how many were found?
F1-score	Harmonic mean of precision and recall	Balances precision and recall
Confusion matrix	Counts correct and incorrect predictions by class	Shows which classes are being confused
ROC-AUC	Area under the ROC curve	Measures ranking ability across probability thresholds; closer t

For this dataset, Scikit-learn labels benign as the positive class (class 1). The classification report also shows separate metrics for malignant and benign. Always state which class is treated as positive when interpreting precision and recall.

Confusion matrix layout: rows represent actual classes and columns represent predicted classes. The diagonal cells are correct predictions; off-diagonal cells are errors.

8. Complete Python Implementation

Copy the following code into one Google Colab or Jupyter Notebook cell. It loads the dataset, trains both models and prints all comparison metrics.

```
# WEEK 3: CLASSIFICATION PROJECT
# Train and compare Logistic Regression and Random Forest

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score,
    ConfusionMatrixDisplay
)

# 1. Load the dataset
dataset = load_breast_cancer(as_frame=True)
X = dataset.data
y = dataset.target

# 2. Explore the dataset
print("Dataset shape:", X.shape)
print("\\nFirst five rows:")
print(X.head())
print("\\nClass distribution:")
print(pd.Series(y).map(
    {0: "malignant", 1: "benign"}
).value_counts())
print("\\nMissing values:", X.isnull().sum().sum())

# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 4. Define two classifiers
models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(
            max_iter=5000, random_state=42
        ))
    ]),
```

8. Complete Python Implementation (continued)

```
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42,
        class_weight="balanced"
    )
}

# 5. Train, predict and evaluate both models
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1,
        "ROC-AUC": auc
    })

    print("\\n", "=" * 55)
    print(name)
    print("=" * 55)
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1-score :", round(f1, 4))
    print("ROC-AUC  :", round(auc, 4))
    print("\\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("\\nClassification Report:")
    print(classification_report(
        y_test, y_pred,
        target_names=dataset.target_names
    ))

    # Display the confusion matrix
    ConfusionMatrixDisplay.from_predictions(
        y_test, y_pred,
        display_labels=dataset.target_names,
        cmap="Blues"
    )
    plt.title(name + " - Confusion Matrix")
    plt.tight_layout()
    plt.show()

# 6. Compare model metrics in a table
comparison = pd.DataFrame(results)
print("\\nFINAL MODEL COMPARISON")
print(comparison.round(4).to_string(index=False))

# 7. Plot metric comparison
comparison.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"]
].plot(kind="bar", ylim=(0, 1.05), figsize=(10, 5))
plt.title("Classification Model Comparison")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

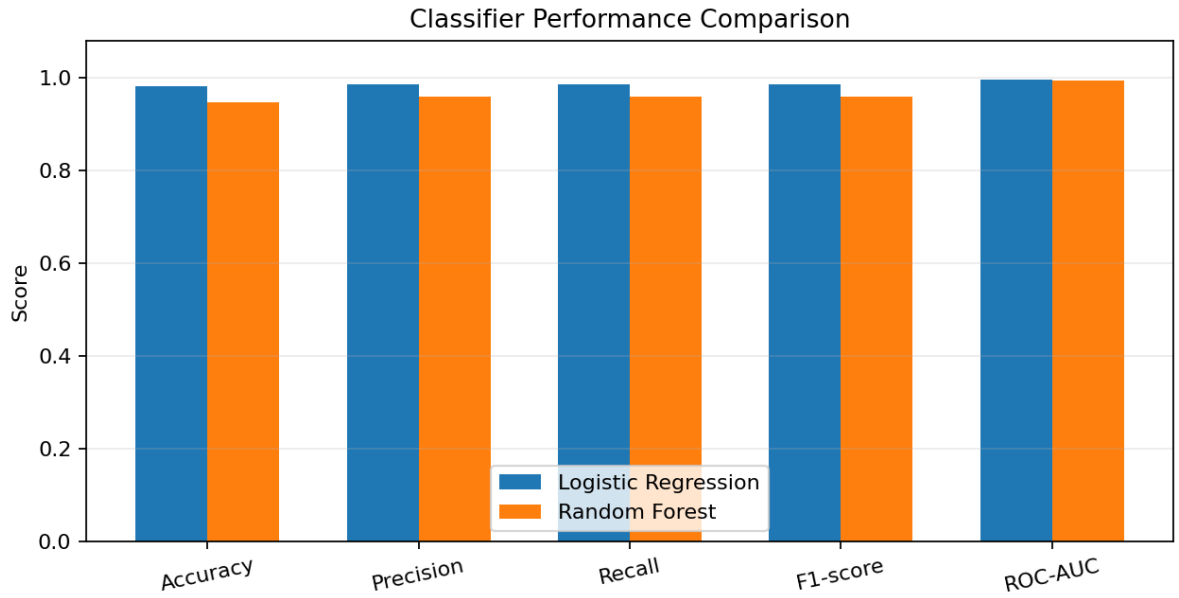
# 8. Select the model with the highest F1-score
best_model = comparison.loc[
    comparison["F1-score"].idxmax(), "Model"
]
print("\\nBest model by F1-score:", best_model)
```

9. Actual Results from This Run

The following results were produced by executing the code with Scikit-learn's built-in dataset, an 80:20 stratified split, random_state=42, Logistic Regression with StandardScaler, and a 200-tree Random Forest. Small differences can

occur with different library versions or settings.

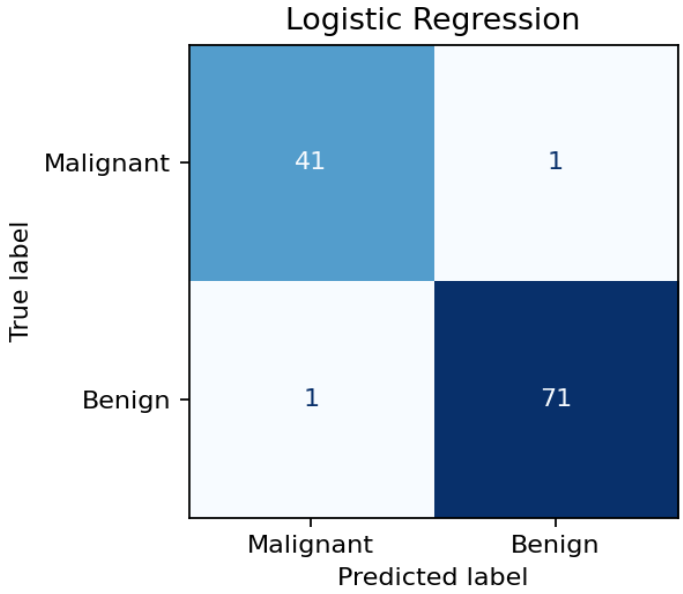
Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.9825	0.9861	0.9861	0.9861	0.9954
Random Forest	0.9474	0.9583	0.9583	0.9583	0.9937



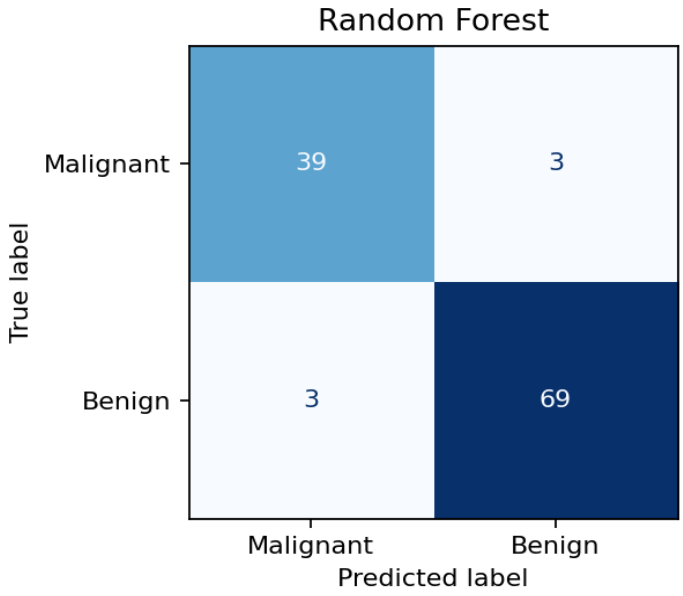
Interpretation: In this run, Logistic Regression achieved an accuracy of approximately 98.25% and an F1-score of approximately 0.9861. Random Forest achieved an accuracy of approximately 94.74% and an F1-score of approximately 0.9583. Logistic Regression performed better on these selected test metrics. This does not prove that it will always be better on every dataset.

10. Confusion Matrix Results

The confusion matrices below show the actual number of correct and incorrect predictions. In this project, the display labels are malignant and benign.



Logistic Regression: 41 malignant records and 71 benign records were correctly classified. One malignant record was predicted benign and one benign record was predicted malignant.



Random Forest: 39 malignant records and 69 benign records were correctly classified. Three malignant records were predicted benign and three benign records were predicted malignant.

Note: In a real medical setting, the cost of a false negative can be serious. Model evaluation must therefore consider class-specific recall, clinical validation, calibration, fairness and expert oversight—not accuracy alone.

11. Model Comparison and Discussion

Comparison point	Logistic Regression	Random Forest
Learning approach	Linear probability-based classifier	Ensemble of decision trees
Feature scaling	Used in this pipeline	Not required here
Accuracy in this run	98.25%	94.74%
F1-score in this run	0.9861	0.9583
ROC-AUC in this run	0.9954	0.9937
Main advantage	Simple and interpretable baseline	Can capture complex non-linear patterns
Main limitation	May underfit non-linear patterns	More complex and less interpretable

Which model is better? Logistic Regression is the better-performing model for this particular split based on accuracy, F1-score and ROC-AUC. Random Forest also performs strongly, but has slightly lower scores in this experiment.

Why can a simpler model win? A complex model is not automatically better. If the relationship in the data is captured well by a simpler decision boundary, Logistic Regression can generalise very effectively.

Fair comparison: Both models used the same training and test records. The scaler was fitted only on the training set through a Pipeline, helping prevent data leakage.

12. How to Run the Project in Google Colab

• Open https://colab.research.google.com/ and create a new notebook.

• Rename it as Week3_Classification_Project.

• Paste the complete Python code from Section 8 into a code cell.

• Click the Run button. The first execution may take a short time while libraries load.

• Check the printed dataset shape, missing-value count, metrics and classification reports.

• Scroll down to view both confusion matrices and the comparison bar chart.

• Save the notebook in your Google Drive and download it as an .ipynb file if required.

No external CSV download is required because the dataset is included in Scikit-learn.

13. Testing Checklist

• The notebook runs from the first cell to the last cell without errors.

• The dataset shape is 569 rows and 30 feature columns.

• The training and testing sets are created before model training.

• Both models are trained using X_train and y_train only.

• Predictions are made using X_test.

• Accuracy, precision, recall, F1-score and ROC-AUC are printed for both models.

• Both confusion matrices and the comparison chart are displayed.

• The conclusion is based on the measured results, not a guess.

14. Conclusion

This project successfully demonstrates a complete supervised classification workflow. The Breast Cancer Wisconsin dataset was loaded and explored, then split into training and testing data. Logistic Regression and Random Forest classifiers were trained and evaluated using multiple metrics. In the recorded experiment, Logistic Regression achieved the higher accuracy and F1-score and was the stronger model on this test split. The project shows why comparing more than one metric is important when selecting a classifier.

Future improvements: Try cross-validation, hyperparameter tuning, feature selection, class-specific metric analysis, calibration and testing on a separate external dataset. These steps can provide a more reliable assessment than one train-test split.

15. Short Submission Note

I completed the Week 3 Classification project by training two machine learning classifiers—Logistic Regression and Random Forest—using the Breast Cancer Wisconsin dataset. I split the data into training and testing sets, evaluated both models using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices, and compared their results. Logistic Regression performed better in this experiment. I learned how to build, test and compare classification models using Python and Scikit-learn.

16. Viva Questions and Answers

1. What is classification?
Classification is a supervised learning method that predicts a category or class label.

2. What is supervised learning?
Learning from examples that contain input features and known target labels.

3. What is a feature?
An input variable used by a machine learning model to make a prediction.

4. What is a target variable?
The output label that the model is trained to predict.

5. Which dataset is used?
The Breast Cancer Wisconsin Diagnostic dataset available in Scikit-learn.

6. Which two algorithms are trained?
Logistic Regression and Random Forest.

7. Why is the data split?
To train on one portion and evaluate performance on unseen test records.

8. What does stratify=y do?
It preserves approximately the same class proportions in train and test sets.

9. What is StandardScaler?
A preprocessing method that centres features and scales them to unit variance.

10. Why use a Pipeline?
It joins preprocessing and model training and helps prevent data leakage.

11. What is accuracy?
The proportion of all predictions that are correct.

12. What is precision?
The proportion of predicted positive cases that are truly positive.

13. What is recall?
The proportion of actual positive cases correctly identified.

14. What is F1-score?
The harmonic mean of precision and recall.

15. What is a confusion matrix?
A table showing correct and incorrect predictions for each class.

16. What is ROC-AUC?
A measure of how well a model ranks positive examples above negative examples across thresholds.

17. Why use Random Forest?
It combines multiple decision trees and can learn non-linear patterns.

18. Which model performed better in this run?
Logistic Regression, based on the reported accuracy, F1-score and ROC-AUC.

19. Does high accuracy guarantee a perfect model?
No. We must also check class-wise precision, recall, F1-score and possible errors.

20. Can this model diagnose a real patient?
No. It is only an educational project and is not validated for clinical use.

17. References

• Scikit-learn documentation — Breast cancer dataset: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

• Scikit-learn documentation — Logistic Regression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

• Scikit-learn documentation — Random Forest Classifier: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

• Scikit-learn documentation — Model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

• Pandas User Guide: https://pandas.pydata.org/docs/user_guide/

Final submission suggestion: Submit this PDF report together with the executed Google Colab/Jupyter notebook and screenshots showing the output metrics, confusion matrices and comparison graph. If your internship portal asks for only one file, follow its upload instructions.